

TrueSight DAO × Tools for the Commons

DAZ Treasury On-Ramp — Tech Spec & Integration Requirements

Prepared by: Sophia Truesight (TrueSight DAO autopilot) · for the Tools for the Commons tech team (Layon Costa, Breno) · **Aug 2026**

Re: 100 FounderHaus Farm QR codes → on-ramped to the DAO treasury on the DAZ system, with receipts per on-ramp transaction, and tree-planting fund management (Paloma).

1 · Purpose & Context

Gary Teh (governor) asked the Tools for the Commons tech team (Layon Costa, Breno) to build an API surface for the **DAZ platform** (daz.toolsforthecommons.com — the Próspera ZEDE treasury/entity stack) and share it with the DAO once ready.

The concrete integration Gary wants:

1. **100 QR-coded cacao tea bags associated with FounderHaus Farm** — a SunMint initiative (truesight.me/programs/founderhaus-farm/ — "100 bags of cacao tea, 100 trees", one bag = one tree).
2. **On-ramp proceeds to the DAO treasury on the DAZ system** as the bags are sold.
3. **Receipts associated with each on-ramp transaction** (per-bag sale → receipt → DAZ transaction).
4. **Fund management for Paloma (FounderHaus, Florianópolis)** — parts of the wallet will be used to buy and plant trees as bags get sold.

This document inventories the DAO's open-source components (Google Apps Scripts, dao protocol "Edgar", Agroverse), pulls in the current integration contracts that matter for this build, and states the requirements the TFTC team should align on during the Monday call.

2 · System Inventory — where everything lives (open source)

Component	What it is	Where it lives
Edgar (dao protocol)	FastAPI contribution server — receives RSA-signed event payloads, verifies signatures, logs to Google Sheets, fires GAS webhooks. Canonical event catalog at `GET /events-catalog`.	`github.com/TrueSightDAO/dao_protocol` · live at `https://edgar.truesight.me`
dao_client	Python client library + CLI (`truesight-dao-*`) — same signing/payload as the browser DApp.	`github.com/TrueSightDAO/dao_protocol` (package `truesight-dao-client`)
DApp	Browser reference implementation of every signed page (scanner, sales, receipts, QR update...).	`github.com/TrueSightDAO/dapp_beta` → beta.dapp.truesight.me; prod dapp.truesight.me

Component	What it is	Where it lives
Google Apps Scripts (GAS)	~50 script projects: ledger processing, QR code generation/web service, sales, repackaging, tree planting, treasury publisher. Reference layout in `google_app_scripts/`; canonical clasp trees in `clasp_mirrors/<scriptId>`.	`github.com/TrueSightDAO/tokenomics`
Agroverse shop	E-commerce static site; loads inventory from agroverse-inventory JSON.	`github.com/TrueSightDAO/agroverse_shop_prod` → agroverse.shop
agroverse-inventory	Generated store/partner inventory + currencies JSON; repackaging composition files.	`github.com/TrueSightDAO/agroverse-inventory` (raw URLs consumed by shop)
treasury-cache	Machine-published treasury snapshots (`dao_offchain_treasury.json`, schema v5) + per-ledger managed-ledger files.	`github.com/TrueSightDAO/treasury-cache`
lineage-assets	Per-QR provenance manifests (`qrs/<qr-id>.json`) — the QR provenance surface.	`github.com/TrueSightDAO/lineage-assets`
SunMint	Farmer tree-planting app (`[TREE PLANTING EVENT]`, offline queue) + photo archive.	`sunmint_beta` / `sunmint_prod` (app), `sunmint` (photos)
TrueChain	Private PoA blockchain audit trail — registries incl. `SalesReceiptRegistry` (receipt hash per sale).	`github.com/TrueSightDAO/TrueChain`

3 · Current Integration Contracts (what exists today)

3.1 Event submission — the universal pattern

Every DAO action is an **RSA-2048 / SHA-256 signed payload** POSTed to `https://edgar.truesight.me/dao/submit\_contribution` (multipart: `text` + optional `attachment`). No API key — **the digital signature IS the authentication**.

Canonical payload shape:

...

[EVENT NAME]

- Label: value
- Label: value

My Digital Signature: <base64 SPKI public key>

Request Transaction ID: <base64 RSA signature of the payload above>

This submission was generated using <source URL>

...

Edgar verifies signature → checks the key is registered + ACTIVE → dedups on `Request Transaction ID` → logs to Google Sheets → fires the event's GAS webhook (see §3.4).

3.2 Relevant event types (from the live Edgar catalog)

Event	Purpose	Key labels
`[SALES EVENT]`	Record a QR-coded product sale (QR → SOLD)	Item, Sales price, Sold by, Cash proceeds collected by, Owner email, Stripe Session ID, Shipping Provider, Tracking number, Attached Filename, Submission Source
`[ASSET RECEIPT EVENT]`	Record receipt of a physical DAO asset (with optional receipt attachment)	Currency, Amount, Description, Fund Handler, Attached Filename, Destination Contribution File Location
`[DONATION MINT EVENT]`	Mint a donation pledge QR (e.g. tree pledge)	QR Code, Currency, Donation Amount, Donor Name, Donor Email, Landing Page, Notes, Attached Filename, Destination Contribution File Location
`[CAPITAL INJECTION EVENT]`	External investment into AGL ledgers	Investor Name, Amount, Currency, Ledger, Description
`[QR CODE EVENT]`	Scan/register a QR	QR Code, Action, Location, Attached Filename, Submission Source
`[QR CODE UPDATE EVENT]`	Update QR metadata (status, owner email, manager)	QR Code, Associated Member, New Status, New Email, Stripe Session ID, Shipping Provider, Tracking Number
`[TREE PLANTING EVENT]`	Farmer reports a planted tree (photo + geo)	Tree Count, Location, Species, Planter, Attached Filename, Submission Source
`[TREE PLANTING LINK EVENT]`	Governor links a sold QR → tree planting (flips QR to ASSIGNED_TO_TREE, books ledger fulfillment)	QR Code, SunMint Submission Message ID, Updated by, Submission Source
`[INVENTORY MOVEMENT]`	Transfer custody between known holders (QR stays IN INVENTORY)	Manager Name, Recipient Name, Inventory Item, QR Code, Quantity, Latitude, Longitude, Attached Filename, Destination Inventory File Location

Event	Purpose	Key labels
[CURRENCY CONVERSION EVENT]	Multi-currency conversion between ledgers (e.g. USD→BRL)	Ledger, Source Currency/Amount, Target Currency/Amount, Implied Rate, Conversion Date, Description, Attached Filename, Destination Currency Conversion File Location

Do not conflate: [SALES EVENT] = end-customer sale (QR → SOLD). [INVENTORY MOVEMENT] = custody transfer between known holders (QR stays IN INVENTORY).

3.3 QR codes — identity & provenance

- Serialized QR id format: ``<FARM>_<PRODUCT>_<YYYYMMDD>_<serial>`` (e.g. `2024OSCAR_CT_20260820_1` = Oscar farm, cacao tea, batch date, serial 1). FounderHaus Farm batch shown on the program page uses this pattern.
- Lookup endpoint: `GET https://edgar.truesight.me/agroverse/qr-code-check?qr_code=<id>` — resolves the QR's `landing_page` and 302-redirects.
- Ledger of record: Main Ledger spreadsheet tab **Agroverse QR codes** (22 cols A–V: `qr_code`, `landing_page`, `ledger`, `status`, `farm`, `state`, `country`, `year`, `Currency`, `dates`, `owner email`, `price`, `manager`, `ledger name`...).
- Status enum: `SCHEDULED_FOR_MINTING`, `MINTED`, `WAREHOUSED`, `ON_CONSIGNMENT`, `CACAO_CIRCLE`, `LOST`, `SOLD`, `TREE_PLANTING_FUNDS_TRANSFERRED`, `EXPENSED`, `ASSIGNED_TO_TREE`, `GIFT`.
- Tree side already minted: `FOUNDERHAUS_BOUGAINVILLEA_20260821_1` (SunMint Tree Planting Pledge — QR Code, manager Gary Teh).
- Provenance surface: `lineage-assets/qrs/<qr-id>.json` renders at `truesight.me/qr/?id=<qr-id>`.

3.4 Webhook dispatch (Edgar → GAS)

After a verified submission, Edgar fires the matching GAS webhook from `dispatch.py` (first-match-wins). Relevant entries: SALES → `parseTelegramChatLogs`; ASSET RECEIPT → `processAssetReceiptsFromTelegramChatLogs`; QR UPDATE → `processQrCodeUpdatesFromTelegramChatLogs`; TREE PLANTING LINK → `processTreePlantingLinksFromTelegramChatLogs`; CURRENCY CONVERSION → `parseAndProcessCurrencyConversionLogs`. Each webhook URL is a server-side env var (`DAO_PROTOCOL_WEBHOOK_<KEY>`); when unset, the GAS cron is the fallback. Sales/receipt events also trigger an **inventory snapshot republish** (`agroverse-inventory` JSON).

3.5 Treasury & ledgers

- Off-chain treasury snapshot:** `treasury-cache/dao_offchain_treasury.json` (schema v5) — inventory by currency across every managed ledger, per-manager breakdown, and per-manager `qr_codes[]` (status, currency, `ledger_shortcut`).

- **Managed ledgers:** per-ledger JSON at ``treasury-cache/managed-ledgers/<ledger>.json`` (AGL4, AGL6, AGL8, AGL10, AGL13–15, BEC, PP1, SEF1, TBM...), rendered by the managed-ledger explorer pattern (e.g. `mirim-bahia.truesight.me`).
- **Main Ledger** Google Sheet is the source of truth: ``Currencies``, ``offchain asset location``, ``Shipment Ledger Listing`` (new ledgers register here, incl. required **Program** rollup: `agroverse / sunmint / fundraiser`).
- **Receipts today:** Edgar uploads an attached file to GitHub and the signed payload carries the URL (``Attached Filename` + `Destination ... File Location``). TrueChain's ``SalesReceiptRegistry`` stores a receipt hash (payment method, amount, reference) per sale.

4 · Requirements — DAZ Treasury On-Ramp (FounderHaus Farm, 100 QRs)

R1 — API surface & auth

- TFTC shares the DAZ API details for: wallet/treasury account creation, deposit (on-ramp), transaction listing, balance, webhooks/notifications, and (if available) idempotency + webhook signing.
- Agree on auth model (API key, OAuth client-credentials, or DAZ-signed requests) and how the DAO's operator keys are provisioned.

R2 — Per-bag on-ramp flow

- For each of the 100 FounderHaus Farm QR bags: a sale is recorded in the DAO as ``[SALES EVENT]`` (QR → SOLD; proceeds + owner email captured). The proceeds then on-ramp into the DAZ treasury wallet.
- The DAZ transaction must be **linkable back to the QR**: carry the DAZ transaction ID/reference inside the signed event (recommend a ``DAZ Transaction Reference`` label on the event) and mirror the Edgar event's ``Request Transaction ID`` back into the DAZ record for reconciliation.

R3 — Receipt association (Gary's question: "Can receipts be associated with the on-ramp transactions?")

- **Yes — the existing contract already supports it end-to-end:** the signed event's ``attachment`` is uploaded to GitHub by Edgar, and the payload carries the receipt URL (``Attached Filename` + `Destination Contribution File Location``). The same pattern can carry the DAZ receipt: submit the DAZ receipt PDF/image as the attachment on the on-ramp event, with the DAZ transaction reference in the payload.
- Recommended receipt fields per transaction: DAZ transaction ID, date, amount (fiat + crypto), currency, source rail (PIX / Wise / stablecoin / Stripe), the DAO event ID (``Request Transaction ID``), and the QR code id.
- Optional hardening: hash the receipt into TrueChain (``SalesReceiptRegistry.recordReceipt``) for an immutable audit trail.

R4 — Treasury & ledger mapping

- Decide which DAO ledger(s) the DAZ wallet maps to. Options: (a) a **new managed ledger** (e.g. `founderhaus`) registered in `Shipment Ledger Listing` with Program = `sunmint` or `agroverse`, published via `treasury-cache/managed-ledgers/founderhaus.json` + explorer page; or (b) reuse an existing AGL/Main Ledger line.
- DAZ balance should flow into `dao\_offchain\_treasury.json` so the public treasury pages reflect it.

R5 — Tree-planting fund (Paloma as fund manager)

- Part of each bag's proceeds is earmarked to buy and plant trees (one bag = one tree). Define: **auto-sweep** of a fixed %/amount per sale to a tree-fund sub-account vs manual disbursement.
- Paloma (FounderHaus) needs a fund-manager view: wallet balance, per-bag contributions, disbursements, and a trail from "bag sold" → "funds on-ramped" → "tree planted" ([TREE PLANTING EVENT] / [TREE PLANTING LINK EVENT] → QR flips to `ASSIGNED\_TO\_TREE` → ledger books `Cacao Tree Planted` asset).
- Permission model: DAZ wallet roles for Paloma (manage/approve disbursements) vs Gary (governor, full visibility) — align with the DAO's governor/permission matrix.

5 · Reference — minimal signed event examples

SALE (one bag):

...

[SALES EVENT]

- Item: 2024OSCAR_CT_20260820_1
- Sales price: 24.00
- Sold by: FounderHaus
- Cash proceeds collected by: Paloma
- Owner email: buyer@example.com
- DAZ Transaction Reference: DAZ-<txn-id>

...

Receipt attachment on the on-ramp leg (uses [ASSET RECEIPT EVENT] pattern):

...

[ASSET RECEIPT EVENT]

- Currency: USD
- Amount: 24.00
- Description: FounderHaus Farm bag 2024OSCAR_CT_20260820_1 sale proceeds on-ramped to DAZ treasury; DAZ txn DAZ-<txn-id>
- Fund Handler: Paloma
- Attached Filename: daz-receipt-<txn-id>.pdf

- Destination Contribution File Location:

<https://github.com/TrueSightDAO/.github/blob/main/assets/receipts/daz/<txn-id>.pdf>

...

Tree funding (when the bag's tree is planted):

...

[TREE PLANTING LINK EVENT]

- QR Code: 2024OSCAR_CT_20260820_1
- SunMint Submission Message ID: <msg-id>
- Updated by: Gary Teh

...

6 · Open Questions for the Monday Call

1. **DAZ API surface** — which endpoints exist for treasury/wallet operations, deposits (on-ramp), balances, transactions, and webhooks? Sandbox availability?
2. **On-ramp semantics** — is "on-ramp" a fiat/crypto deposit into a DAZ wallet, a mint of a DAZ-native asset, or both? Receipt format the DAZ side emits?
3. **Reconciliation key** — confirm DAZ transaction reference ↔ Edgar `Request Transaction ID` ↔ QR code as the join key.
4. **Ledger mapping** — new `founderhaus` managed ledger vs reuse of existing AGLs.
5. **Tree-fund mechanics** — auto-sweep % per sale vs manual disbursement; Paloma's role/limits on the DAZ wallet.
6. **Cadence** — how often do on-ramp batches run (per sale / daily / weekly reconciliation)?

7 · Getting Started (for the TFTC team)

- **Integration guide:** `dao\_protocol/INTEGRATION\_GUIDE.md` (signing, payloads, event catalog, webhooks).
- **Live catalog:** `https://edgar.truesight.me/events-catalog`
- **Client:** `pip install truesight-dao-client` → `truesight-dao-auth login --email you@example.com` → submit events.
- **QR check:** `https://edgar.truesight.me/agroverse/qr-code-check?qr\_code=<id>`
- **Program page (context):** `https://truesight.me/programs/founderhaus-farm/`
- **DAZ platform:** `https://daz.toolsforthecommons.com`

Prepared by Sophia Truesight · TrueSight DAO autopilot · admin+sophia@truesight.me